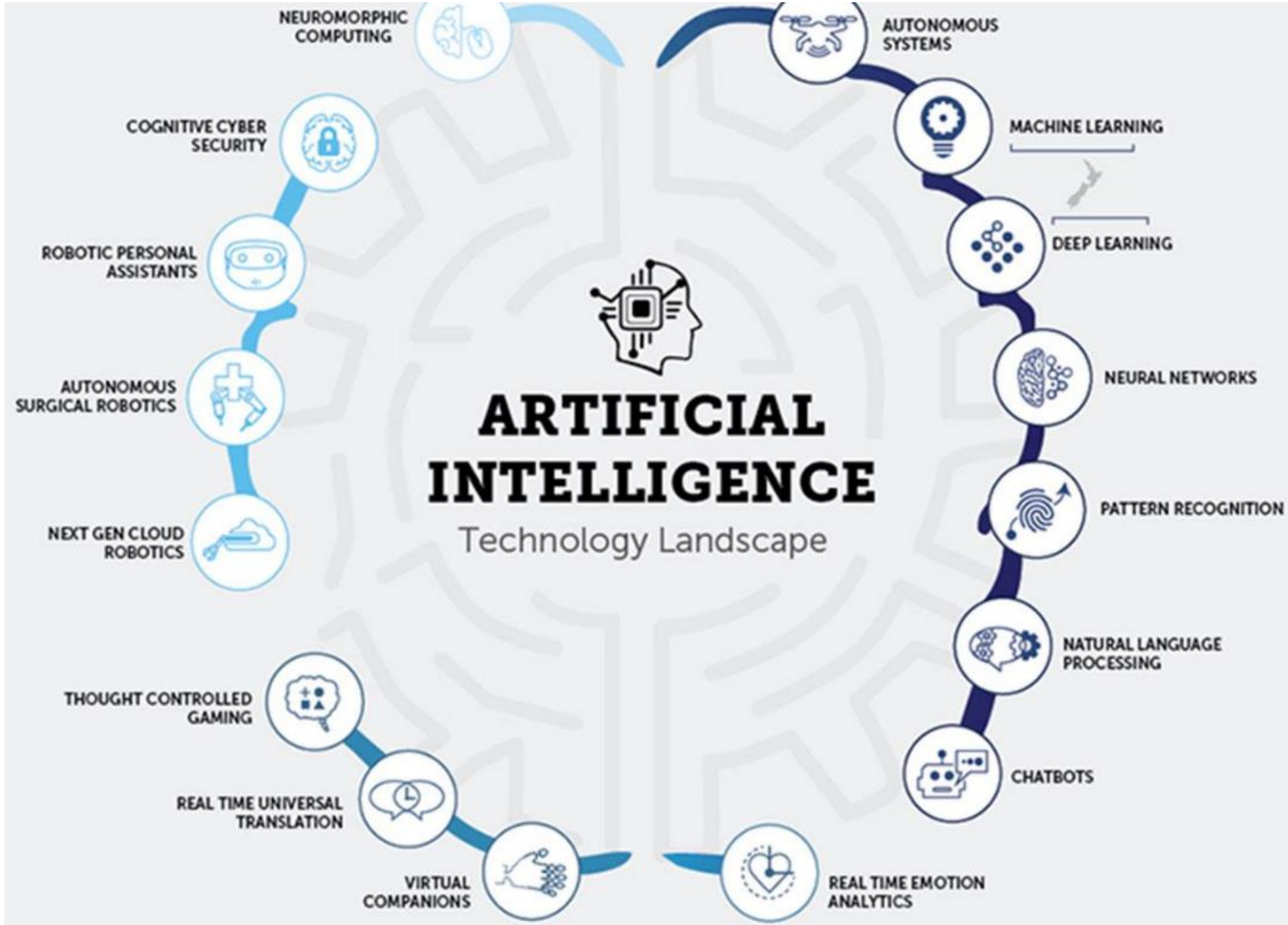
ARTIFICIAL INTELLIGENCE


NEUROMORPHIC COMPUTING
COGNITIVE CYBER SECURITY
ROBOTIC PERSONAL ASSISTANTS
AUTONOMOUS SURGICAL ROBOTICS
NEXT GEN CLOUD ROBOTICS
THOUGHT CONTROLLED GAMING
REAL TIME UNIVERSAL TRANSLATION
VIRTUAL COMPANIONS
ARTIFICIAL INTELLIGENCE
Technology Landscape
AUTONOMOUS SYSTEMS
MACHINE LEARNING
DEEP LEARNING
NEURAL NETWORKS
PATTERN RECOGNITION
NATURAL LANGUAGE PROCESSING
CHATBOTS
REAL TIME EMOTION ANALYTICS

History of AI:

- 1642-1936: Charlie's Analytical Engine
- 1834: Tom Stendage
  : wrote research paper on thinking machine
- 1840: Joseph Faber
  : first talking machine named Euphonia
- 1921: C Zech author-Karel Capek
  : R.U.R[Rossum's Universal Robots]
  : It was science fiction play.
- 1950: Year of Turing Test
  : Alan Turing [Computing Machinery and Intelligence]

History of AI:

- 1957: Neweel and Simon

  : They predicted that one day computers will beat human being in chess.

- 1997: Deep Blue developed by IBM computers

  :Play Chess

  : The game was of 6 match.

  - first match was played in Philadephia and the score was 4(garry Kasparov)-2(deep blue)

  -second match was played in New York and score was 3(deep blue)-2. (1 draw)

Applications of AI:

1. Game Plan:

Reasons: -easy to understand.

-well defined set of rules.

-structured in a way or nature of game is not ambiguous (confusing).

-no financial or ethical burden.

-no expert knowledge is required.

-easy to generate state space.

2. Automated Reasoning and Theorem Proving:

- It is a subfield of automated reasoning and mathematical logic dealing with proving mathematical theorem by computers programs.

Application of AI:

3. Natural Language Processing:

- Understanding: machine understand human language.
- Translation: the language convert 1 medium to 2 medium.
- Generation: subtype of NLP.

4. Robotics:

- A Remotely Operated Vehicle(ROV) is an unoccupied underwater robot that is connected to a ship by a series of cables.
- cabels transmit command and control signals between the operator and the ROV.
- Fully autonomous is a robot that acts without resource to human control. E.g. Roomba
- Semi autonomous is a term for automation that can make decisions and perform action without direction i.e. they need human in certain situations.
- Three types robots: Land, Water, Ariel.

Applications of AI:

5. Expert Systems:

- has knowledge and after match they take decisions.
- Specific and domain knowledge system should open to learn.
- example of expert system:

MYCIN:

- It take sample of a blood and finds the bacteria that causes the disease in the body.

DENDRAL:

- It is chemical analysis expert system used in organic chemistry to detect unknown organic molecules.

PROSPECTOR:

- It was used by geologist to find minerals at particular area by taking samples.

Applications of AI:

6. Neural Networks: (Part of deep learning)

- Neural Network is also known as artificial neural network (ANNs).
- ANN is based on a collection of connected units or nodes called artificial neurons, which loosely model the neurons in a biological brain.
- Biological neural network (BNN) is a network of neurons that are connected together by axons and dendrites.
- The connections between neurons are made by synapses.

Task Domain of AI:

1. Mundane task: (Repetative task)

- Perception

  -speech

  -vision

- Natural Language Processing

  -Translation

  -Generation

- Reasoning

  -Common sense

  -Interation

Task Domain of AI:

2. Formal task
   - Problem solving
   - Mathematical perspective
3. Engineering task
   - designing intelligent system
     e.g. ADAS in MG (partial automation)
   - fault finding
     e.g. autonomous car
   - expert analysis (prediction)
   - it is not limited. It require knowledge that many of us do not have
     e.g. in car- engine analysis, tyre analysis

Characteristic of AI:

- It is voluminous (size).
- It is constantly changing.
- It is hard to characterize.
- Its rules will change based on data.
- It is very dynamic.
- Choice of intelligent.

Turning Test

- The Turing Test is a thought experiment that determines if a machine can think like a human. It was developed by Alan Turing, a mathematician, computer scientist, and cryptanalyst, in 1950. The test is still used to study artificial intelligence (AI) and understand how machines interact with humans.

HOW IT WORKS?

- A human evaluator judges a conversation between a human and a machine.
- The evaluator tries to identify the machine.
- The machine passes if the evaluator can’t reliably tell the difference.

IMPORTANCE:

- The Turing Test is important because it helps us understand how machines interact with humans, and what it means to be intelligent or think. It also helps us understand the capabilities of AI, which is becoming more integrated into our lives.

Turing Test:

During the Turing Test, the human interrogator asks several questions to both players. Based on the answers, the interrogator attempts to determine which player is a computer and which player is a human respondent.

Player A
Computer

Player B
Human Responder

Player C
Interrogator

- Question to Respondents
- Answers to Question

abc NEWS

AI Techniques

- AI technique is a method that exploits knowledge(using knowledge in a strategic way) that should be represented in such a way:

  - the knowledge captures generalization

  - It can be modified to decrease error and to adopt the changes in the environment.

    e.g. tic tac toe

Characteristic of AI techniques:

1. It can be used in many situations even it is not totally accurate or correct.
2. It can be used to overcome its own sheer-bulk by narrowing the range of possibilities.

AI Techniques

Tic-Tac-Toe

Strategy 1:

Algorithmn:

Step 1) View the vector board as a ternary number and convert it to a decimal number.

Step 2) Use the number computed in (Step 1) an index into home table and excess the vector store them.

Step 3) The vector selected in a (Step 2) represent the way the board will look after the move that should be made.

So, set the board equalto the vector.

AI Techniques

Advantages	Dis-advantages
-You can play all the moves.	-requires lot of space.
-optimized move	-requires a lot of time to prepare move table
-possibilities of update on move table	-It is not scalable.
-keep an eye on opponent move	-Hard work is require to prepare such a move table.

AI Techniques

Strategy 2:

Algorithmn: (three subprocedures)

- **MAKE2**
  - Checks if the center square (square 5) is blank.
  - If the center square is blank, returns 5.
  - If the center square is not blank, returns any blank non-corner square (positions 2, 4, 6, or 8).
- **POSSWIN(p)**
  - Determines if player p (where p is either 3 for X or 5 for O) can win on the next move.
  - Checks each row, column, and diagonal to see if it contains a possible winning move.
  - Multiplies the values of the squares in each line:
  - If the product is 18 (3 * 3 * 2), then X can win.
  - If the product is 50 (5 * 5 * 2), then O can win.
  - If a possible winning line is found, identifies the blank square in that line and returns its position.
  - Returns 0 if no winning move is possible.
- GO(n)
  - Make a move in square ‘n’.
  - Sets ‘BOARD[n]’ to 3 if the current player is X or 5 if the current player is O.
  - Increments ‘TURN’ by 1.

Three Sub – procedures:

Make2	Returns 5 if the center square of board is blank (board[5]=2). Otherwise, the function returns any blank non-corner square (2,4,6 or 8).
Posswin(*p*)	Returns 0 if the player *p* cant win on his next move; otherwise returns the number of the square that constitutes a winning move. This function will enable us to win and to block opponent's win.
Go(n)	Makes a move in square n. This procedure sets board[n]=3 if turn is odd, or 5 if turn is even. It also increments turn by 1.

- Odd numbered moves – X
- Even numbered moves – O

Turn=1 (X)	Go(1) (upper left corner).
Turn=2 (O)	If board[5] is blank, Go(5), else Go(1).
Turn=3 (X)	If board[9] is blank, Go(9), else Go(3).
Turn=4 (O)	If Posswin(X) is not 0, then Go(Posswin(X)) [i.e. block opponent's win],else Go(make2).
Turn=5 (X)	If Posswin(X) is not 0, then Go(Posswin(X)) [i.e. win] else Posswin(O) is not 0, then Go(Posswin(O))[i.e. block win], else if board[7] is blank , then Go(7) , else Go(3).
Turn=6 (O)	If Posswin(O) is not 0, then Go(Posswin(O)) [i.e. win] else Posswin(X) is not 0, then Go(Posswin(X))[i.e. block win], Else Go(Make2).

Turn=7 (X)	If Posswin(X) is not 0, then Go(Posswin(X)) [i.e. win] else Posswin(O) is not 0, then Go(Posswin(O))[i.e. block win], else go anywhere that is blank.
Turn=8 (O)	If Posswin(O) is not 0, then Go(Posswin(O)) [i.e. win] else Posswin(X) is not 0, then Go(Posswin(X))[i.e. block win], Else go anywhere that is blank.
Turn=9 (X)	Same as turn=7.

- The program is not efficient in terms of time (as it has to check several conditions before making move), more efficient in terms of space.
- It is easier to understand the program’s strategy or change strategy if desired.
- But, it still can not generalize to 3 dimensions.

Strategy – 2’

8 3 4 1 5 9 6 7 2	•A magic square – all rows, columns and diagonals sum to 15. •This simplify process of checking for a possible win. •We keep a list, for each player , of squares in which he has played. •To check a possible win- consider each pair of squares (x,y) owned by that player compute the difference z=15-(x+y). If square z is blank, a move there will produce win.

Strategy – 3
Min – Max Procedure

Data structures-

BoardPosition	A structure containing a 9-element vector representing the board, a list of board positions that could result from the next move, And a number representing an estimate of how likely the board position is to lead to win for the player.

AI Techniques

Strategy – 3

Min – Max Procedure

Algorithms:

1. See the each possible move.

2. See if it is a win. If so, call it the best by giving it the highest possible rating.

3. Otherwise, consider all the moves the opponent could make next. See which of them is worst for us. Assume the opponent will make that move.

4. The best node is then the one with the highest rating.

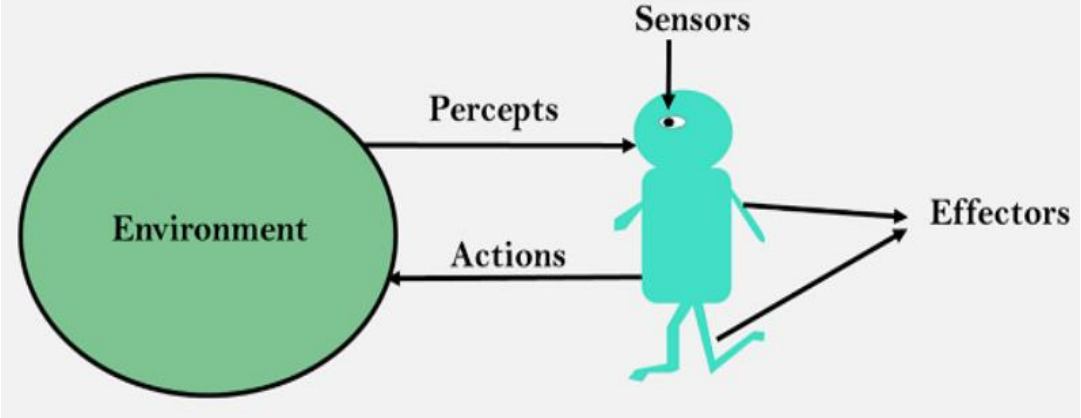
Comments-

- This program will require much more time but it is superior to other programs –

1. It could be extended to handle games more complicated than Tic-tac-toe for which other programs fall apart.
2. It can be augmented by knowledge about games. For example- instead of considering all possible next moves, it might consider only subset of them (determined by simple algorithm)
3. Program 3 is example **AI technique**. For small problems, it is less efficient than more direct methods. However, it can be used in situations where those methods would fail.

Intelligent Agents

- Intelligent Agent: An intelligent agent is an autonomous entity which act upon an environment using sensors and actuators for achieving goals. An intelligent agent may learn from the environment to achieve their goals. A thermostat is an example of an intelligent agent.
- Agents: An agent can be anything that understand its environment through sensors and act upon that environment through actuators. An Agent runs in the cycle of **perceiving**, **thinking**, and **acting**.



- A **human agent** has sensory organs such as eyes, ears, nose, tongue and skin parallel to the sensors, and other organs such as hands, legs, mouth, for effectors.
- A **robotic agent** replaces cameras and infrared range finders for the sensors, and various motors and actuators for effectors.
- A **software agent** has encoded bit strings as its programs and actions.

Intelligent Agents

- **Sensor:** Sensor is a device which detects the change in the environment and sends the information to other electronic devices. An agent observes its environment through sensors.
- **Actuators:** Actuators are the component of machines that converts energy into motion. The actuators are only responsible for moving and controlling a system. An actuator can be an electric motor, gears, rails, etc.
- **Effectors:** Effectors are the devices which affect the environment. Effectors can be legs, wheels, arms, fingers, wings, fins, and display screen.

Rationality:

- The rationality of an agent is measured by its performance measure. Rationality can be judged on the basis of following points:

- Performance measure which defines the success criterion.
- Agent prior knowledge of its environment.
- Best possible actions that an agent can perform.

Nature of Environment

- About task environments, which are essentially the “problems” to which rational agents are the “solutions.”
- We had to specify the performance measure, the environment, and the agent’s actuators and sensors. We group all these under the heading of the task environment. For the acronymically minded, we call this the PEAS (Performance, Environment, Actuators, Sensors) description. In designing an agent, the first step must always be to specify the task environment as fully as possible.

Agent Type	Performance Measure	Environment	Actuators	Sensors
Taxi driver	Safe, fast, legal, comfortable trip, maximize profits	Roads, other traffic, pedestrians, customers	Steering, accelerator, brake, signal, horn, display	Cameras, sonar, speedometer, GPS, odometer, accelerometer, engine sensors, keyboard

Do it Yourself:

Agent Type	Performance Measure	Environment	Actuators	Sensors
Vaccum cleaner	Time taken to clean, Battery efficiency, Coverage of the area, Noise Level	Boundaries (walls, stairs, edges), Rooms or floors (carpet, tile, wood, etc.)	Wheels (for movement), Indicators (lights or sounds to notify status), Brushes and suction motor (to clean dust and debris)	Dirt detection sensor, Obstacle detection sensor (infrared, ultrasonic), Battery level sensor

Nature of Environment

1. Fully observable(e.g. Chess) or Partial observable(e.g. Poker)
2. Deterministic(e.g. sudoku) or Stochastic(e.g. Rolling Dice)
3. Episodic(e.g. image classification tasks) or Sequentially(e.g. navigating a vehicle)
4. Static(e.g. Static puzzle) or Dynamic(e.g. real time strategy game)
5. Discrete(e.g. board games with a fixed number of moves) or continuous(e.g. robotic control tasks with continuous movement)
6. Single agent(e.g. a single-player video game) or Multi agent(e.g. a multiplayer game or stock trading)

Do it Yourself:

1. Observable vs. Partially Observable

If the vacuum cleaner can sense the entire environment (e.g., a robot with cameras and sensors covering all areas), the environment is fully observable.

If it can only sense local surroundings (e.g., a basic model that detects dirt only where it is), it is partially observable.

2. Deterministic vs. Stochastic

If the outcome of an action is always predictable (e.g., moving left always results in moving left), the environment is deterministic.

If the result of actions is uncertain due to external factors (e.g., suction failing, obstacles moving), the environment is stochastic.

3. Static vs. Dynamic

If the environment does not change while the vacuum is deciding its next action, it is static.

If the environment can change independently (e.g., people walking in and making the floor dirty again), it is dynamic.

4. Discrete vs. Continuous

If the vacuum cleaner operates on a grid-based model with limited positions and actions, the environment is discrete.

If movement and sensing happen in a continuous space (like in real-world robotic vacuums), the environment is continuous.

Do it Yourself:

5. Episodic vs. Sequential

- If actions do not depend on past actions (e.g., each cleaning task is independent), the environment is episodic.
- If past actions affect future decisions (e.g., remembering cleaned areas to avoid redundancy), the environment is sequential.

6. Single-agent vs. Multi-agent

- If the vacuum cleaner is the only intelligent entity, the environment is single-agent.
- If there are other intelligent agents (e.g., multiple robots working together or a pet reacting to the vacuum), the environment is multi-agent.

Structure of Agents:

- The job of AI is to design an agent program that implements the agent function—the mapping from percepts to actions. We assume this program will run on some sort of computing device with physical sensors and actuators—we call this the architecture:

agent = architecture + program

where,

Architecture is machinery that an AI agent executes on

Agent function is used to map a percept to an action

Program is an implementation of agent function. An agent program executes on the physical architecture to produce function

- Obviously, the program we choose has to be one that is appropriate for the architecture. If the program is going to recommend actions like Walk, the architecture had better have legs.

- **Problem-solving agents** are a type of artificial intelligence (AI) agent designed to achieve specific goals by systematically exploring possible actions and states to find a solution. These agents are capable of formulating a problem, searching for a solution, and executing actions that lead to a desired outcome. E.g. Game playing, Route planning, Automated planning and Scheduling.
- **Knowledge-based agents** are a type of artificial intelligence (AI) agent that utilizes a structured knowledge base to make decisions, reason about the world, and take actions. E.g. Expert system, NLP, Robotics.

Chapter 2

Problems as State Space Search

- State space search is a fundamental concept in artificial intelligence (AI) and computer science, used to define and solve problems in terms of exploring different possible states or configurations of a system.

1. State Space Representation

State: A state represents a configuration or condition of the system at a particular point in time. It is a snapshot of all relevant variables or aspects of the problem at a given moment.

Initial State: The starting point of the problem, representing the condition of the system before any actions have been taken.

Goal State: The desired outcome or configuration that the system should reach. A problem can have one or multiple goal states.

2. Actions

Actions: These are the possible operations that can be applied to move from one state to another. Actions are the "moves" or "steps" that can be taken to progress towards the goal state.

Transition Model: Describes the rules that define how actions transform one state into another. For every action applied to a state, there is a resulting state.

Problems as State Space Search

3. State Space

The state space is the set of all possible states that can be reached by applying the available actions starting from the initial state. It is often visualized as a graph where nodes represent states and edges represent actions.

4. Path of Actions

A path in the state space is a sequence of actions that leads from the initial state to a goal state. Solving the problem involves finding a path from the initial state to one of the goal states.

5. Search Strategies

To find a solution, different search strategies are applied to explore the state space. These strategies can be categorized into uninformed (blind) search methods, like breadth-first search or depth-first search, and informed (heuristic) search methods, like A* or greedy search.

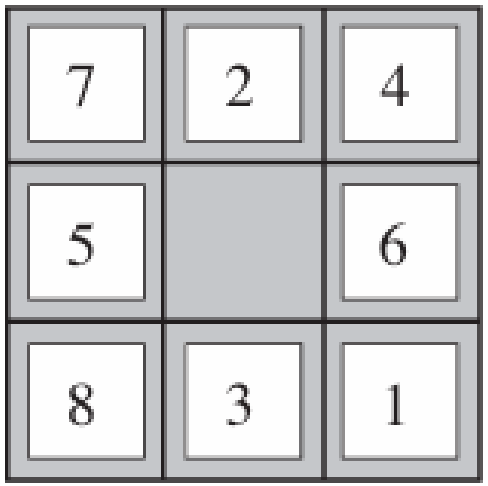
6. Solution

The solution to the problem is the sequence of actions (or the path) that leads from the initial state to a goal state, typically with the minimum cost or the most optimal route, depending on the problem.

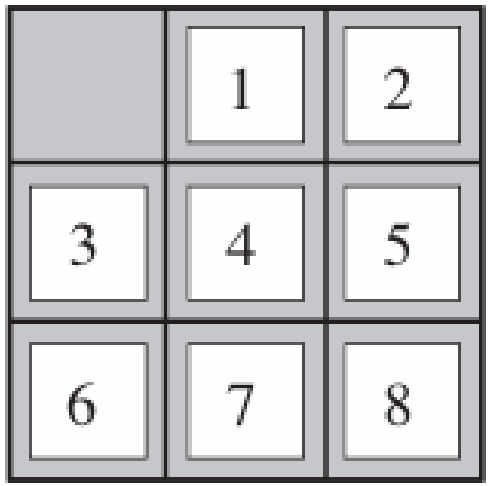
Example: Solving the 8-Puzzle

- **States:** Each state is a different configuration of the tiles on the 3x3 grid.
- **Initial State:** The starting configuration of the puzzle.
- **Goal State:** The configuration where all tiles are in the correct order.
- **Actions:** Moving a tile into the empty space.
- **Transition model:** Given a state and action, this returns the resulting state; for example, if we apply Left to the start state, the resulting state has the 5 and the blank switched.
- **Search:** Explore the state space to find the sequence of moves that leads from the initial to the goal state.

Example: Chess, Tic-Tac- Toe



Start State



Goal State

Example: Solving the Chess

- **States:** A state represents the current arrangement of pieces on the chessboard.
- **Initial State:** The standard initial chessboard setup.
- **Goal State:** One player achieves checkmate, forcing the opponent's king into an inescapable position.
- **Actions:** Moving a piece according to its allowed movement rules.
- **Transition model:** Moving a piece results in a new board state, possibly capturing an opponent’s piece.
- **Search:** AI explores possible future moves using techniques like Minimax with Alpha-Beta Pruning.

Example: Solving the Tic – Tac - Toe

- **States:** A state is the current placement of "X" and "O" on the 3×3 grid.
- **Initial State:** An empty 3×3 grid before any moves are made.
- **Goal State:** A player aligns three marks (X or O) in a row, column, or diagonal.
- **Actions:** Placing an "X" or "O" in an empty square.
- **Transition model:** Placing a mark updates the board state.
- **Search:** AI often uses the Minimax algorithm, which can explore all possible move sequences due to the small state space.

Three Water Jug Problem

- In this we have three jugs of capacity 8, 5 and 3 litres. The intial state will be (8,0,0) and you to reach the goal state (4,4,0). The puzzle may be solved in optimal steps, passing through the following sequence of states.

Production System

1. A production system is a model of computation used in AI for implementing search algorithm and for modelling human problem-solving process.

2. It provides control over problem solving process and it consists of:

(i) A set of rules called over production rules consisting of condition-action pair.

(I) Condition is a pattern which determines when a rule may be applied to the object.

(II) Action, it determines the problem-solving step.

(ii) One or more knowledge that contain whatever information is appropriate particular task. Some parts of the knowledge may be permanent, while other parts of it may assume to be the solution of the current problem.

3. A control strategy is a mechanism which loops in a simple cycle to decide which rule can be applied.

(i) It should be simple, not too complex.

(ii) It should not loop infinity.

4. Rule applier [T, B, L, R].

e.g. 8 puzzle game moves.

Heuristic

- Heuristic come from geek word heurika which means I found something.

Heuristic -> search algo. -> find direction or path -> best path or optimal -> to reach goal state

- Heuristic improves the efficiency of a search process possibly by sacrificing the claims of completeness. i.e. it leads near to goal state but not a goal state.
- A heuristic is a technique that is used to solve a problem faster than the classic methods. These techniques are used to find the approximate solution of a problem.
- Best state: in which the cost is less.
- Goal state: in which the cost is more but first we have seen.
- The move of taking the number to the goal state position is known as cost.
- We have initial state and going to goal state we need cost and it should be less if cost is decreasing then we can say that we are in correct direction.

Heuristic for 8-puzzle problem

Algorithm:

1. Find the distance of each tile from its goal position.
2. Find the sum of all distances and reject the one has greatest distance.
3. Choose the possibility which minimize the distance in order to select a tile for making the next move, use the following steps:

3.1 Consider each tile from the current state that can be move say t1,t2,……,tk.

- Make the move and get the resulting states say S1,S2,…..Sk.

3.2 Evaluate each state S1, S2, ……,Sk.

3.2.1 In each state Si consider each tile.

3.2.2 Find its distance from the destination position in the goal state and add this distances for each tile.

3.2.3 Let the resulting distances be D1, D2,……., Dk.

3.2.4 Find the smallest distances Di out of all distances to make the next move.

Problem Characteristic

- Is the problem decomposable? (divide in parts or not)

  No or Yes
- Can the solution step be ignored or undone?

  i) Recoverable ii) Irrecoverable iii) ignore
- Is the problem universe is predictable? (opponent move is predictable or not)
- Is the good solution absolute(fix) or relative(dynamic)?
- Is the solution a state or path? (we can decide the route)
- What is role of knowledge? (level of knowledge)

  less, moderate or high
- Does the task require interaction with person?

  Conversational or Solitary(1 player)

Example of Problem Characteristic

- Tic-Tac-Toe

1. No 2. Irrecoverable 3. not predictable 4. Absolute 5. State (fix board position 6. Less 7. Conversational

- 8-queen problem

1. Yes 2. Irrecoverable 3. predictable 4. relative 5. Path 6. High

7. Conversational

- Travelling Salesman Problem

1. No 2. Recoverable 3. Predictable 4. Relative 5. Path 6. Moderate

7. Solitary

DO IT YOURSELF

1. Chess 2. Tower of Hanoi 3. Water Jug Problem 4. 8-Puzzle Problem

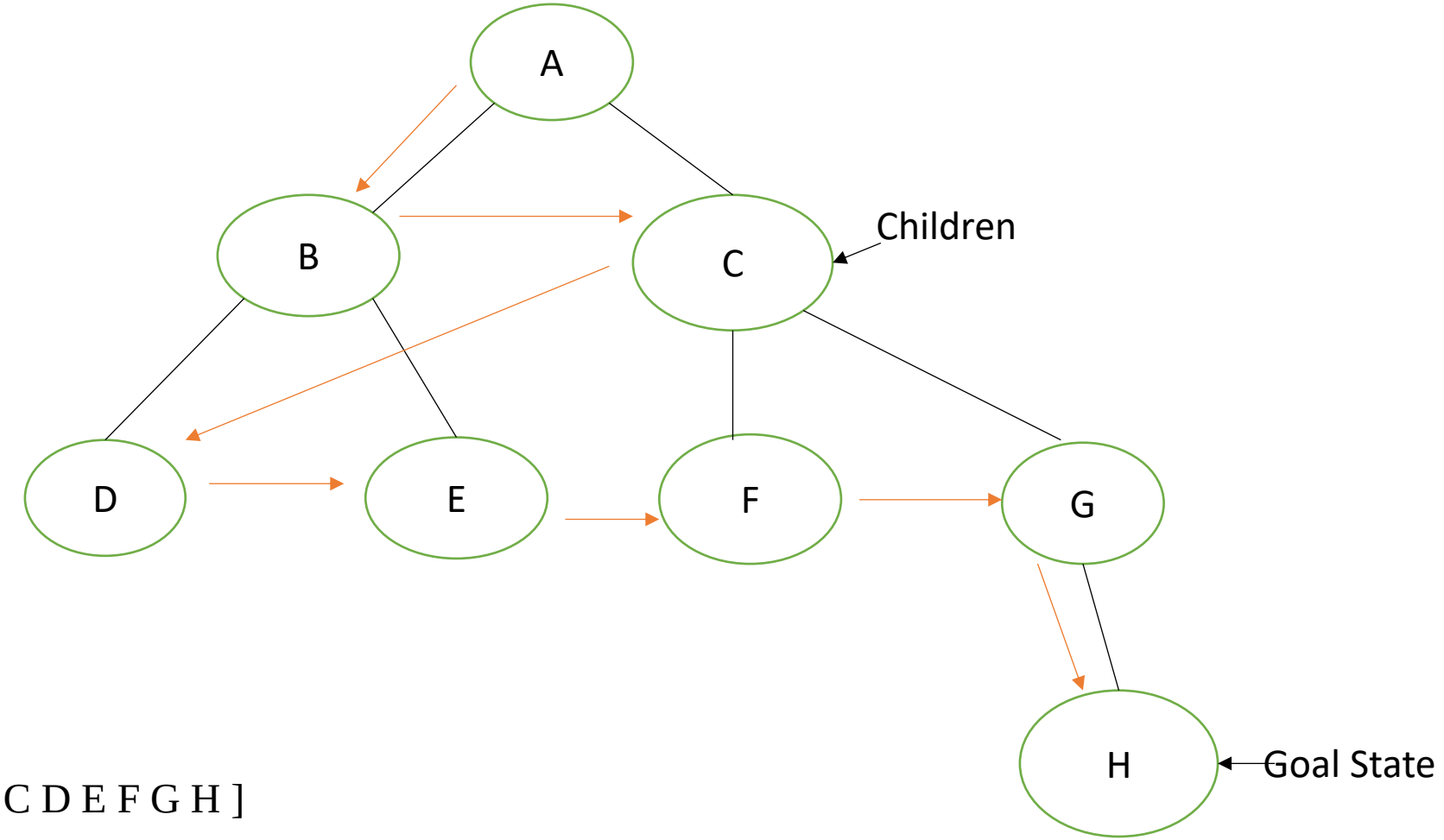
Breadth First Search

- BFS explores all possible states (or nodes) level by level, starting from a given initial state, and is particularly useful for finding the shortest path in an unweighted graph or tree.

Algorithm:

- **Step 1:** SET STATUS = 1 (ready state) for each node in G
- **Step 2:** Enqueue the starting node A and set its STATUS = 2
- **Step 3:** Repeat Steps 4 and 5 until QUEUE is empty
- **Step 4:** Dequeue a node N. Process it and set its STATUS = 3
- **Step 5:** Enqueue all the neighbours of N that are in the ready state (whose STATUS = 1) and set their STATUS = 2
- **Step 6:** EXIT

Breadth First Search



Path: [A B C D E F G H]

Games

- 8 – queen problem
- Farmer, fox, goat, grass problem
- 3 missionaries and 3 cannibals problem

Uninformed and Informed Search

- **Uninformed Search:** This method, also known as blind search, does not use additional information and includes techniques like Breadth-First Search and Depth First Search.
- **Informed Search:** These methods use extra information to decide the next steps. They are more cost-effective and efficient than uninformed methods.

Best first Search

- In Best First Search we use an evaluation function to decide which among the various available nodes is the most promising (or ‘BEST’) before traversing to that node.
- BFS uses the concept of a Priority queue and heuristic search.

Algorithm:

1. Start with OPEN(priority queue) containing just the initial state.

2. Until a goal is found or there are no nodes left on OPEN do;:

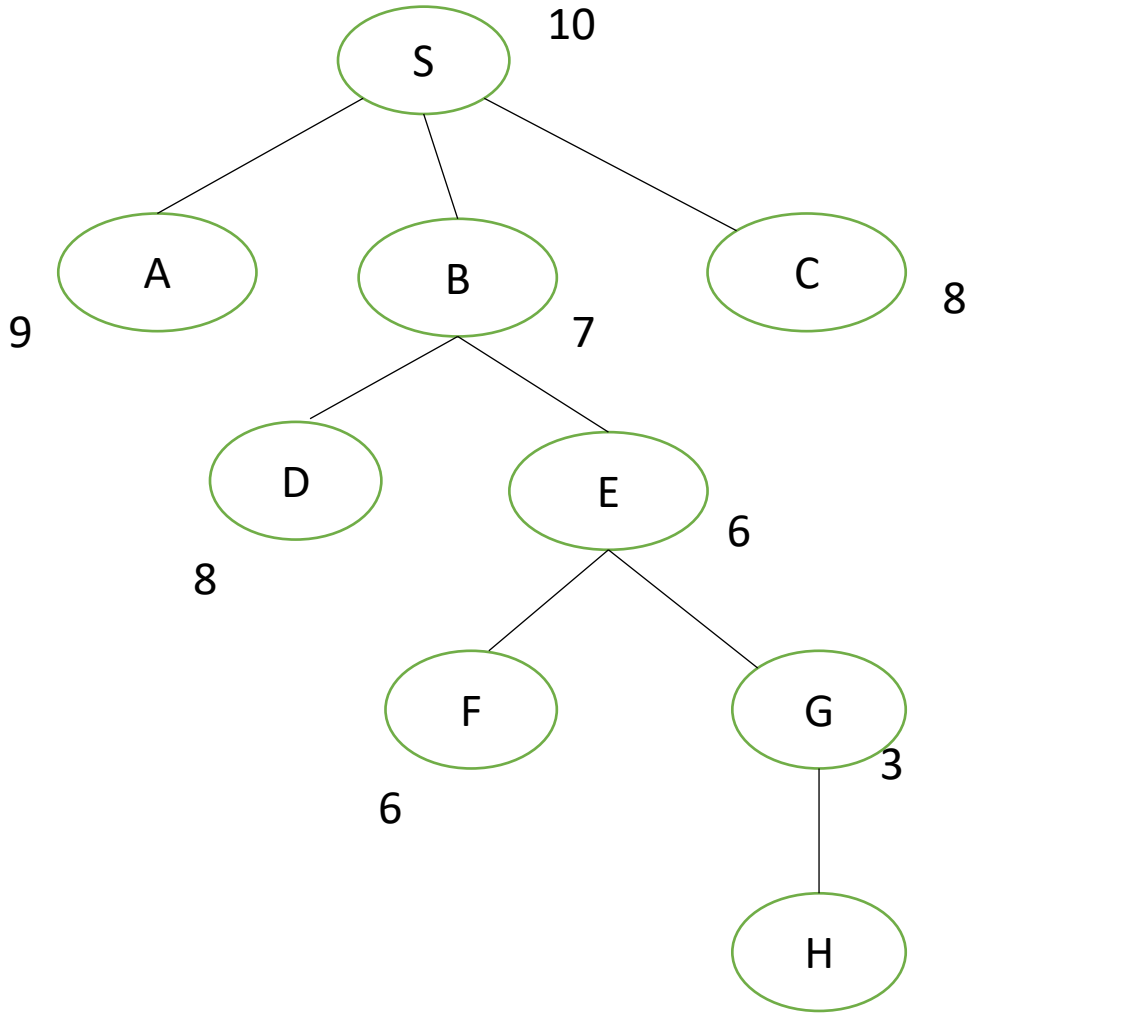
   (a) Pick them best node on OPEN.

   (b) Generate its successors.

   (c) For each successor do:

      (i) If it has not been generated before, evaluate it, add it to OPEN, and record its parent.

      (ii) If it has been generated before, change the parent if this new path is better than the previous one. In that case, update the cost of getting to this node and to any successors that this node may already have.


S
10
A
9
B
7
C
8
D
8
E
6
F
6
G
3
H
0
goal state